ISSAT Sousse

Republic of Tunisia

University of Sousse

Research Master's Degree in Computer Science — Intelligent Pervasive Systems

Coding Agents Evaluation

Application to Software Architectures

Measuring SOLID-principle adherence across five coding agents and two languages, over 1,413 real merged pull requests

Prepared by

Akram Ben Fekih

Supervised by

Mrs. Maha Khemaja

Academic Year 2025 / 2026

Agenda

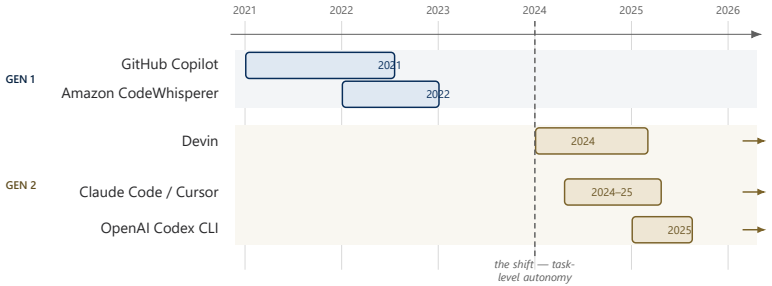
1. Context and problem
2. State of the art
3. SOLID: the principles, and why
4. Experiment and workflow
5. Findings and results
6. Conclusion

Key finding. One question runs through all six parts: does the code these agents write *and get merged* hold up against the structural design standards the field already has — SOLID — and can that be measured, not just asserted?

01

Context and Problem

From autocomplete to autonomous agents



First-generation tools suggested the **next line**; second-generation **agents** plan, edit, run tests, and revise — **without** a human in the loop, following the ReAct (reason + act) pattern.

That is a change in *kind*, not degree — why function-level benchmarks stop being sufficient.

Vibe coding, and its riskier cousin

Vibe coding: the developer keeps intent and direction; the agent handles implementation.

Vibe architecting (Konrad et al., 2026): structural decisions shaped implicitly by *how a prompt happens to be worded*, not by deliberate design.

Their evidence: the *same* task, different prompts, produced systems from **141 lines / 2 files** to **827 lines / 6 files** — no architecture decision records, no rationale.

Agent-authored code is not hypothetical: the AIDev dataset (Li et al., 2025) alone catalogues **789,364 merged** pull requests from autonomous agents, and it is already being studied at scale (Cynthia et al., 2026).

Key finding. Speed and pass-rate are one narrow slice of quality. SOLID, code smells, and architectural metrics exist precisely because functional correctness and good design are different things.

The benchmark–reality gap

HumanEval / SWE-bench (Jimenez et al., 2024) ask: does the code pass tests? Nothing about structure, coherence, or maintainability.

METR (2026) (METR, 2026): ∼half of SWE-bench-Verified passes would **not** be merged by the repository's own maintainers — a ∼24-point gap, roughly **2**× the human reference rejection rate.

SlopCodeBench (SprocketLab, 2026): quality erodes across successive checkpoints; agent code runs ∼2.2× more verbose than comparable open source; the best checkpoint solve rate was **17.2%**.

NITR (Zhu et al., 2026a): **13.3%** of AI-generated code passes every functional test while failing an architectural-quality check — a *structural oracle failure* invisible to test suites.

Research objectives

O1. Build a repository-scale framework measuring all **five SOLID principles** on **real merged pull requests**, via deterministic structural proxies.

O2. Turn construct validity into a **measured quantity** — validate every proxy against an independent reading judge (precision/recall), not an assumption.

O3. Apply the framework to **five leading agents** — Claude Code, Copilot, Cursor, Devin, OpenAI Codex — pooled and repository-mix-controlled.

O4. Locate **where risk concentrates** (inside one unit vs. between units), and test whether **local ML** beats a deterministic proxy where its precision ceiling is genuinely semantic.

Research questions

RQ1 Do agents differ in SOLID adherence on real merged PRs, and where do the differences concentrate?

RQ2 Can SOLID be measured at scale, how faithfully do the proxies agree with a reading judge, and with independently built analysers?

RQ3 Is risk intra-unit or inter-unit; can local ML beat a deterministic proxy; and what actually ports to a second language?

Key finding. RQ1 is answered from the **Python** corpus only. AIDev supplies almost no Java from any agent but OpenAI Codex, so the Java arm is a within-agent replication, not a second comparison.

02

State of the Art

Foundations: three layers of design quality

Design principles

Martin's SOLID (Martin, 2000) — five heuristics for where responsibility, extension, substitutability, and dependency should sit.

Code smells

Fowler's taxonomy (Fowler et al., 1999) — God Class, Feature Envy, Data Clumps — a shared vocabulary for *what* is wrong.

Architectural metrics

CK suite (Chidamber and Kemerer, 1994) and Martin's *A*/*I*/*D* (Martin, 2003) — the quantitative vocabulary that makes design measurable.

These layers are not independent — SOLID violations manifest as smells, and smells are what CK-style metrics are calibrated to catch. This thesis works at the **principle** layer, because it is the most design-relevant and least reliably measured of the three.

What the literature already shows

Pehlivan et al. (2025) — prompting LLMs to *detect* SOLID violations: SRP/ISP detected reliably; **DIP is consistently the hardest**; accuracy falls as code complexity rises.

NITR (Zhu et al., 2026a) — agents succeed on dependency-control tasks only **4.3%** of the time, on responsibility-decomposition only **15.2%**.

Zhu et al. (2026b) — a *machine signature* of AI defects (broken abstractions, systematic duplication, over-engineering); a **Volume-Quality Inverse Law**: code volume predicts structural degradation almost perfectly, and neither test coverage nor prompt detail stops it.

"Beyond Functional Correctness" — large AI-IDE projects that pass their tests still carry SRP-shaped smells.

Key finding. The common thread: passing tests and being well-designed are different

Where the literature leaves a gap

Scale, with attribution

ProjDevBench: 20 problems (Lu et al., 2026). SWE-bench: issue-level, not design. This thesis: **546 Python PRs / 147 repos** + **867 Java PRs / 27 repos**, each attributable to one agent.

Depth + measured validity

Most studies *assert* their metric stands in for the principle. This thesis **measures** the gap: precision/recall against a reading judge, for all five principles.

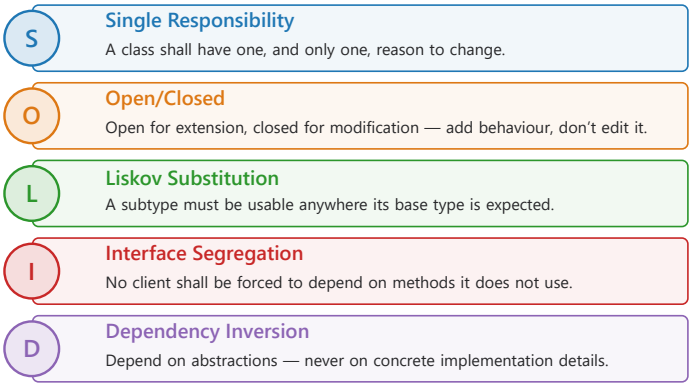
Agent-level specificity

The literature varies the *model*, holding scaffolding constant. This thesis isolates the **agent** — Claude Code, Copilot, Cursor, Devin, Codex — as the unit of comparison.

03

SOLID: the Principles, and Why

The five SOLID principles



Why a validated proxy, not an assumed one

SOLID (Martin, 2000) is the most cited normative OO design framework — and, per the literature, the dimension **least reliably measured at repository scale**.

None of the five principles is directly observable in source text: each is a statement about *intent* — the reasons a module may change, the variants a design anticipates, the promises a subtype makes.

This study's stance (**post-positivist**): treat every measurement as a *proxy*, and **quantify the proxy-to-principle gap** — precision/recall against an independent reading judge — rather than assert fidelity.

Key finding. Every number here is a statement about the proxy first, and about the principle only *through* it — why Part 5 reports agreement scores beside every rate.

04

Experiment and Workflow

Research design: three scoping decisions

1. **Real merged PRs**, not freshly generated repositories — ecological validity, and each PR is a bounded unit attributable to one agent.
2. **Two languages**: Python first (richest zero-config tooling), then **Java** — the language where the sharpest objection to the whole design ("these principles are an awkward fit outside mandatory OOP") can be tested directly.
3. **Deterministic proxies**, validated against a reading judge. An LLM is used *only* to read source and label a validation sample — never to compute a reported score.

JavaScript was also curated and measured, then deferred on sample grounds — only 554 merged PRs qualify and 411 come from a single agent.

The corpus

Python — the cross-agent comparison

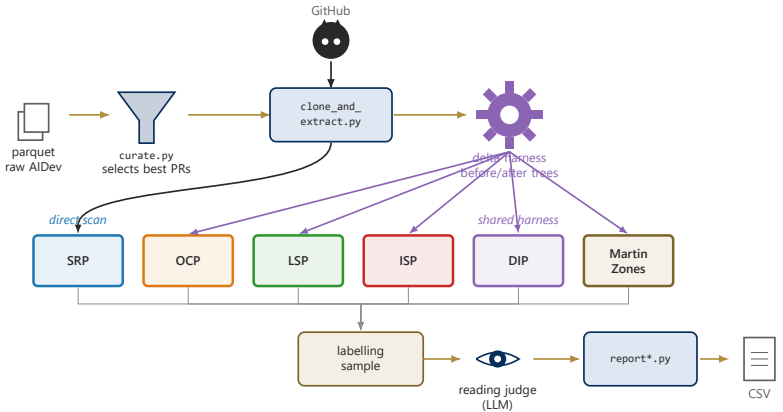
Agent	PRs	
Claude Code	37	smallest
Copilot	150	capped
Cursor	81	
Devin	128	
OpenAI Codex	150	capped
Total	**546**	147 repos

Java — replication + depth

Agent	Eligible
OpenAI Codex	**3,024**
Cursor	104
Devin	34
Claude Code	8
Copilot	**0**

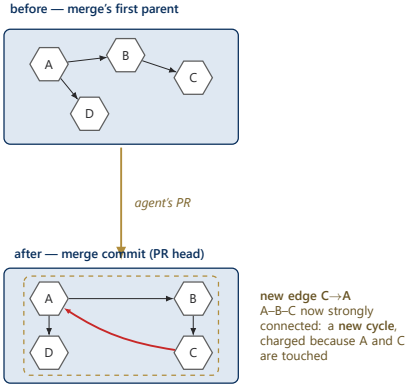
Java: 867 PRs / 27 repos curated — a **within-agent** study of Codex, by dataset constraint, not choice.

The analysis pipeline



Every stage is **independently resumable**. Core footprint: `pandas`, `pyarrow`, `lizard`, `radon`, `git` — plus **JavaParser + SymbolSolver** for Java.

The before/after delta harness



For the relation-level principles (DIP, ISP, OCP, LSP), only the difference the PR introduces is charged — both trees are fetched, and **only the delta** counts.

Authorship $\neq$ **authorship-adjacent editing**: a class the agent *created* is scored apart from one it merely touched.

A violation is charged if **either side** of the relation sits in a touched file — editing a base class charges the subclasses it breaks, even unopened ones.

Principle → smell → proxy

Principle	Forbidden smell	Structural proxy	Primary signal
SRP	God Class	size/complexity rubric + cohesion suppressor	LCOM4; C3 cohesion
OCP	type/tag switch	dispatch width + closed-set suppressor	branch count; closed-type-set rule
LSP	non-substitutable override	AST override compatibility (mypy's rule)	signature/variance; refused bequest
ISP	fat interface / forced stubs	interface width + implementer stub ratio	abstract-method count; stub ratio
DIP	concretion dependency / cycle	before/after import-graph delta	Tarjan cycles; Martin *D*

Every proxy fires on a structure a violation *tends* to leave behind, never on intent itself.

The suppressors — exact definitions

SRP — LCOM4 (cohesion) and C3 (the semantic suppressor).

LCOM4: one node per non-constructor method, connected if two share an attribute or call each other — **LCOM = connected components.** C3: each method → a smoothed TF-IDF vector v_i; mean pairwise cosine similarity ($C3 \geq 0.4$ un-flags the class):

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|} \qquad C3 = \underset{i<j}{\text{mean}} \ \cos(v_i, v_j)$$

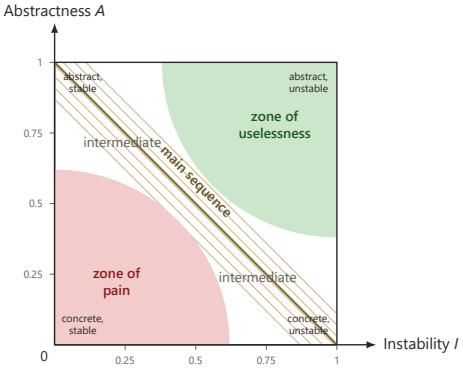
OCP — the closed-set suppressor.

A chain is flagged only if wide *and* not over a closed set:

$$\text{smell} = (\text{branch_count} \geq \text{threshold}) \ \wedge \ \neg\, \text{is_closed_type_set}(\text{types})$$

`is_closed_type_set`: every type a Python builtin, **or** sharing one fixed grammar prefix (e.g. `ast.*`) — exhaustive by construction.

Martin's plane — and the sixth, descriptive evaluator



$I = C_e/(C_e + C_a)$ instability; $A = N_a/N_c$ abstractness (share of abstract classes).

$D = |A + I - 1|$ — distance from the "main sequence" good packages sit near.

Zone of pain: concrete & stable — everything depends on it, it can't change. **Zone of uselessness**: abstract, used by nobody.

Key finding. No delta, no violation flag: validation scored the *D*-as-change signal at precision **0.000**. Package *position* stays a sound descriptor; package *change* does not.

Validating the instrument

A **reading judge** (an LLM, Claude) labels a stratified sample — flagged and unflagged, blind to the rubric's own verdict — by reading the actual source.

Three of five proxies (SRP, OCP, LSP) start as **high-recall screens**; a **suppressor** then removes a *nameable* false-positive idiom without cost to recall.

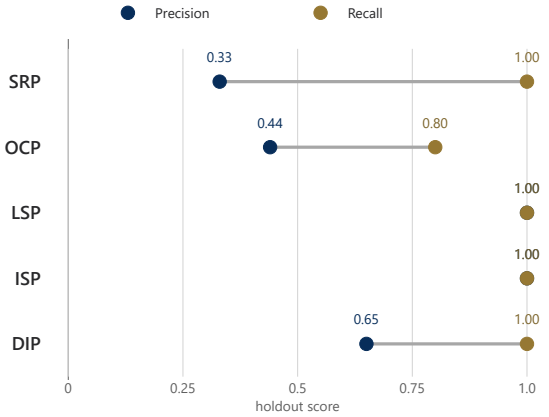
Every candidate change passes a **validation gate**: it must raise precision at the same recall, on a sample **it was not tuned on**.

Every comparative result is reported **pooled** and **macro-averaged per repository** — macro leads, because one repository can otherwise masquerade as an agent property.

05

Findings and Results

Instrument validation — Python



ISP, LSP → crisp detectors after their suppressors (holdout P = 1.00).

SRP, OCP, DIP → high-recall screens (P = 0.33–0.65) — useful to *triage*, not to *convict*.

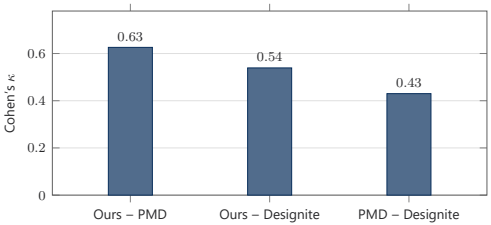
DIP's 0.65 hides a total split: `new_cycle` **13/13**, `mean_d_worse` **0/7**.

Instrument validation — Java, and a second axis

Principle	P / R / F1	Reading
SRP	**0.455** / 1.000 (**0.679** production)	best SRP precision of either language; holdout did *not* collapse
LSP	0.500 / 1.000	aggregate hides a total split (next slide)
ISP	0.462 / 1.000	width is the weak half; errors are wide-but-cohesive contracts
OCP	0.364 / **0.533**	only detector whose *recall* also suffers
DIP	1.000 / 1.000	2 flagged PRs — a mechanism, not a rate

Key finding. A second, judge-independent axis: agreement with three published Java analysers (CK, PMD, DesigniteJava) run over the *same* source trees.

Convergent validity: our rubric vs. published tools



Size/complexity converge strongly: $\rho = 0.976$ (WMC), 0.947 (LOC), 0.966 (methods); **91.0%** of our flags sit in CK's own top decile.

Flag-level: $\kappa = 0.626$ vs. PMD, 0.539 vs. DesigniteJava.

The two established tools agree with each other less ($\kappa = 0.430$) than either agrees with us.

686/875 (78.4%) of our flags corroborated by $\geq$1 independent tool.

Key finding. Our screen is not an outlier — it sits inside the spread published Java god-class detectors already show among themselves, and closer to each of them than they are to each other.

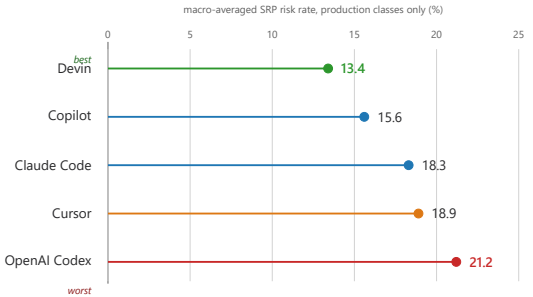
Java: one signal aggregate can conceal three different truths

LSP-Java signal	Flagged	TP	Precision
`refused_bequest`	8	8	**1.000**
`precondition_strengthened`	11	3	0.273
`postcondition_weakened`	3	0	**0.000**

The aggregate 0.500 is the average of a **perfect** detector, a mostly-noisy one, and a **broken** one — every `refused_bequest` flag is a genuine `throw new UnsupportedOperationException` over a base that does real work.

Key finding. Finding 7: report precision *per signal* wherever a detector combines signals. A composite figure is a summary statistic, not a property of the instrument.

Agent differentiation — SRP (Python)



Rate shown is **macro-averaged per repository**, **production classes only** — pooled, Codex looks *best* because 52% of its classes come from one repository.

Test classes are **34.7%** of the corpus, and agents differ ∼4× in how much test code they write (Claude 48.6% vs. Copilot 11.6%) — test classes flag at ∼1/3 the production rate.

Under both controls, **Codex stays worst**; Copilot's earlier "best" position was a test-writing artefact, not a design one.

Agent differentiation — OCP, and the honesty of small counts

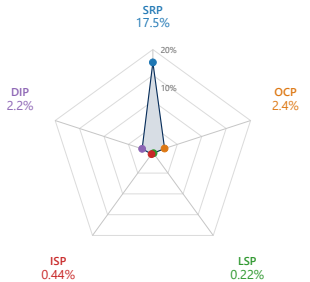
Over 452 analysable Python PRs: **Cursor 5/70, Devin 3/112, Claude Code 2/34, Copilot 1/85, Codex 0/150.**

The entire OCP signal is eleven pull requests. A single PR moves an agent by several points — counts are printed beside every rate for exactly this reason.

ISP, LSP, DIP are near-zero at PR level: ISP flags **2**, LSP flags **1**, DIP flags **10** (5 real cycles) — across the *same* 452 PRs.

Key finding. The qualitative contrast is what's robust, not any ranking built on these counts: intra-unit principles (SRP, OCP) produce a real per-PR signal; inter-unit principles (DIP, ISP, LSP) essentially do not.

Finding 1 — risk is intra-unit, not inter-unit



Python corpus, 452 analysable PRs — each principle's own unit (SRP: macro class-risk rate; others: % of PRs flagged).

SRP spikes far out; OCP/DIP register faintly; ISP/LSP barely leave the centre.

Key finding. Java replicates it in the language most likely to break it: of 867 PRs, every cross-unit principle is below 1% (OCP 5, ISP 3, DIP 2, LSP 1) — Java mandates OOP, so "the language doesn't force structure" is not the explanation.

Deterministic rules beat off-the-shelf machine learning

Experiment	Type	Result	Shipped
LSP FP suppressor	deterministic	holdout P **0.42** → **1.00**, recall kept	✓
OCP closed-set rule	deterministic	holdout P **0.33** → **0.44**, recall kept	✓
SRP embedding-C3	embedding	god 0.934 vs. cohesive 0.954 → recall collapsed	×
OCP branch-cohesion	embedding	premise inverted (FPs score *higher* than smells)	×
SRP classifier	trained model	no gain at R≈1.0; only re-weights size	×

An off-the-shelf code encoder (GraphCodeBERT) compresses source into a narrow ∼0.85–0.99 cosine band that **drowns** the exact axis that matters — shared vocabulary (SRP), open-vs-closed type set (OCP). TF-IDF out-separates it.

Three more cross-cutting findings

Finding 4 — porting is not re-parsing. Three of Python's five LSP signals **cannot occur** in code `javac` accepts (invariant parameters, no narrowing). The Java detector needed a different *signal set*, not just a re-parsed one.

Finding 5 — Codex writes tests, not architecture. Of 439 new Java classes Codex authored, **422 (96.1%) are tests**; of 17 new production classes, **0 are flagged** as a god class — while 12.9% of the 976 production classes it *touches* are, and every worst offender **predates** the PR.

Finding 8 — half the corpus has nothing to invert into. At package level, **median abstractness** $A = 0.000$ **in both languages**: half of all placed packages declare no abstract class, interface, or protocol at all.

06

Conclusion

Answering the research questions

RQ1 Agents differ, but only on **intra-unit** principles — roughly 8 macro points on SRP (Devin best, Codex worst); OCP shows a real but tiny signal (11 PRs). Inter-unit principles show ≈0 per-PR signal for everyone.

RQ2 Yes, measurable — at **two** operating points: crisp detectors (ISP, LSP after suppressors, P≈1.0) and high-recall screens (SRP, OCP, DIP, P = 0.33–0.68). Java's independent cross-check places the class instrument *inside* the spread published tools show among themselves.

RQ3 Risk is **intra-unit**, replicated in Java; deterministic rules beat off-the-shelf ML on every gated experiment; portability is bounded by **language semantics** (what the compiler enforces) before it is bounded by tooling.

Contributions

C1. A **validated, repository-scale instrument** for all five SOLID principles, in two languages, with precision/recall reported rather than assumed.

C2. An **explicit map** of where structural SOLID measurement works (crisp detectors) and where it only screens (semantic ceilings) — falsifiable, not a single pooled number.

C3. An **empirical account** of where agent-authored risk concentrates: inside single units, not across them — replicated in a language chosen to break the claim.

C4. A **gated ML-vs-deterministic comparison**, with the negative ML result reported, not dropped.

C5. A **demonstrated language port**: what carries over (pipeline shape, two metric cores) and what needed re-deriving (thresholds, one principle's signal set).

Practical implications

Choosing between agents: SOLID risk is a *second axis* beside functional pass-rate — Devin has the lowest God-Class rate among the five once repo mix and tests are controlled; Codex's Java output is conservative in a specific sense (96% of new classes are tests).

Building a review gate on top of an agent: an **LSP or ISP** flag can be trusted close to face value; an **SRP, OCP, or DIP** flag should trigger a human look, not an automatic rejection.

The single cheapest, most reliable check available today: `refused_bequest` (P = 1.00, and independently corroborated by developers' own `# type: ignore` comments).

Limitations

Survivorship bias — the corpus is *merged*, permissively-licensed, $\geq$25-star PRs only. This cannot separate "agents rarely err" from "review catches their errors."

The Java arm is one agent (OpenAI Codex) — a property of AIDev's coverage, not a design choice; no cross-language agent ranking is claimed.

Several counts are small — 11 OCP PRs, 10 DIP PRs, 4 Java DIP flags — reported as counts, not smoothed into rates a single PR could move by several points.

Future work

Measure rejected and abandoned PRs — the direct answer to the survivorship-bias limitation, and the single highest-value extension.

Draw the four outstanding Java holdouts (OCP, LSP, ISP, DIP), so every Java figure becomes an out-of-sample estimate, as SRP's already is.

A churn-based, width-independent OCP signal — a class edited across many PRs is, by definition, not closed for modification; both corpora already carry the history.

A module- or function-scoped SRP proxy for repositories where the class is not the dominant unit, motivated by Finding 8's Python package result (48.2% have no class-ratio denominator).

Closing remark

The design faults autonomous coding agents leave in real, merged code are concentrated **inside single units** — a class doing too much, a conditional that should have been an extension point — and almost absent from the **relations between units** that a single change rarely gets to author in full.

That distinction held across five agents and two languages chosen specifically to stress it. What remains open is whether it still holds at the scale where architecture is actually won or lost: a repository maintained over months, by an agent and by the humans who edit its output afterwards.

Thank you

Questions and discussion

Akram Ben Fekih · Supervised by Mrs. Maha Khemaja
ISSAT Sousse — Research Master's in Computer Science, Intelligent Pervasive Systems

Selected references I

Chidamber, S. R. and Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6):476–493.

Cynthia, S. T., Das, J. K., and Roy, B. (2026). Are we all using agents the same way? an empirical study of core and peripheral developers' use of coding agents. arXiv:2601.20106.

Fowler, M., Beck, K., Brant, J., Opdyke, W., and Roberts, D. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. (2024). Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*.

Konrad, P. M., Adam, T. L., Terrenzi, R., and Ayvaz, S. (2026). Architecture without architects: How ai coding agents shape software architecture. arXiv:2604.04990.

Selected references II

Li, H., Zhang, H., and Hassan, A. E. (2025). The rise of ai teammates in software engineering (se) 3.0: How autonomous coding agents are reshaping software engineering. arXiv:2507.15003; AIDev dataset, replication package at github.com/SAILResearch/AI_Teammates_in_SE3 and huggingface.co/datasets/hao-li/AIDev.

Lu, P., Zhang, S., Hou, Y., Ye, L., Huang, C., Chen, Z., Zeng, J., Jiang, H., Liu, P., Wang, Y., and Yang, M.-H. (2026). Projdevbench: Benchmarking ai coding agents on end-to-end project development. arXiv:2602.01655.

Martin, R. C. (2000). *Design Principles and Design Patterns*. Foundational reference for the SOLID principles.

Martin, R. C. (2003). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. Source of the package metrics used for DIP: Instability (I), Abstractness (A), and Distance from the Main Sequence ($D = |A + I - 1|$).

METR (2026). Many swe-bench-passing prs would not be merged into main. METR research note, 10 March 2026.

Selected references III

Pehlivan, F., Ergüzen, A. U., Moslemi Yengejeh, S., Lami, M., and Koyuncu, A. (2025). Are we solid yet? an empirical study on prompting llms to detect design principle violations. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. arXiv:2509.03093; 240 manually validated examples.

SprocketLab (2026). Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks. arXiv:2603.24755.

Zhu, H., Zhang, Q., Wang, J., Yang, Z., and Qiu, Y. (2026a). Needle in the repo: A benchmark for maintainability in ai-generated repository edits. arXiv:2603.27745.

Zhu, Y., Tsantalis, N., and Rigby, P. C. (2026b). Ai-generated smells: An analysis of code and architecture in llm and agent-driven development. arXiv:2605.02741.